

AMD CPU PoC — MariaDB Core-Scaling Benchmark

README & Procedure — amd-mariadb-poc-v1.2.sh

2026-07-22

Contents

Overview	1
What the wrapper adds on top of the base runner	1
Prerequisites	2
Command-line Options	2
Procedure — Running the Benchmark	3
Step 1: Copy the script to the benchmark host	3
Step 2: (Optional) Sanity-check the plan with a dry run	3
Step 3: Run the interactive wizard (recommended for a customer PoC)	3
Step 4: Or run fully non-interactively	3
Step 5: Let it run	4
Step 6: Collect the report	4
PoC Tips	4
Troubleshooting	4
Reference	5

Overview

`amd-mariadb-poc-v1.2.sh` is an interactive, customer-facing Proof-of-Concept runner that demonstrates AMD EPYC CPU core scaling for MariaDB using the HammerDB TPC-C OLTP workload (and optionally TPC-H analytics). It is a wrapper around the base benchmark runner in [skariapaul/mariadb-tpcc-bench](https://github.com/skariapaul/mariadb-tpcc-bench) (auto-cloned on first run), so this single script is the only thing you need to copy onto a benchmark host.

What the wrapper adds on top of the base runner

- **Interactive PoC wizard** — prompts for core counts, NUMA pinning, run sizing, and report options, with sane AMD-aware defaults for every field.
- **AMD-aware host tuning** (optional, needs sudo, auto-reverted on exit):
 - CPU frequency governor → `performance`
 - `vm.swappiness` → `1`
 - Transparent hugepages → `never`, with explicit 2 MiB hugepages sized to the InnoDB buffer pool (the base runner then enables MariaDB `large_pages=ON`)
- **Full system inventory capture** — CPU/NUMA/memory/kernel/BIOS details saved alongside the results for the report.
- **Per-run measurement extras**:
 - CPU utilisation sampling via `mpstat`

- CPU package power via RAPL (`/sys/class/powercap`), wrap-safe sampled every 2 seconds — used to compute NOPM/Watt “performance-per-watt”
- **Scaling-efficiency analysis** — speedup vs. the lowest core count tested, and % scaling efficiency relative to ideal linear scaling.
- **Consolidated report** in Markdown, with an optional PDF rendered via **pandoc** + LaTeX, containing:
 - Executive summary with headline speedup
 - System-under-test inventory table
 - TPC-C scaling table (NOPM, speedup, efficiency %, watts, NOPM/W)
 - ASCII throughput chart
 - Transaction latency percentiles and TPC-H results (if run)
 - Appendix A: every generated `my.cnf`, one per core count
 - Appendix B: raw `lscpu` / memory details

Prerequisites

Run on the benchmark host itself (bare metal or a VM with the cores you want to test).

Ubuntu/Debian

```
sudo apt-get install -y docker-ce docker-compose-plugin git libmariadb3 sysstat numactl
```

for PDF report output:

```
sudo apt-get install -y pandoc texlive-latex-recommended texlive-fonts-recommended
```

RHEL/Rocky

```
sudo yum install -y docker-ce docker-compose-plugin git mariadb-connector-c sysstat numactl pandoc
```

Also required:

- Docker daemon running and reachable (`docker info` succeeds)
- User is in the `docker` group, or can `sudo`
- ~20 GB free disk for container images, datadirs, and results
- (Optional but recommended) `sudo`/root access, for host tuning and readable RAPL power counters

The script itself checks all of this in a pre-flight checklist and will offer to auto-install **pandoc** + LaTeX if missing and `sudo` is available.

Command-line Options

All options are optional — the interactive wizard will ask for anything not supplied on the command line.

Option	Description	Default
<code>--cores "8 16 32 64"</code>	Core counts to test (space-separated)	wizard suggests based on host CPU count
<code>--warehouses N</code>	TPC-C warehouses (recommend $\geq 10 \times$ max cores)	wizard recommends <code>max_cores x 10</code>
<code>--rampup N</code>	Ramp-up minutes per run	2
<code>--duration N</code>	Timed-run minutes per run	10
<code>--tpch</code>	Also run TPC-H analytics	off
<code>--tpch-sf N</code>	TPC-H scale factor	1
<code>--buffer-pool SIZE</code>	InnoDB buffer pool, e.g. 64G	75% of host RAM
<code>--mariadb-ver VER</code>	MariaDB Docker image tag	11.4
<code>--numa-node N</code>	Pin runs to NUMA node N (auto <code>cpuset</code> per core count)	none (or wizard prompt if host has >1 NUMA node)
<code>--no-host-tuning</code>	Skip governor/hugepage/swappiness tuning	tuning enabled
<code>--report md\ pdf\ both</code>	Report format	both

Option	Description	Default
<code>--customer "Name"</code>	Customer name printed on the report cover	“AMD EPYC PoC”
<code>--title "Text"</code>	Report title override	“MariaDB Core-Scaling Benchmark — AMD CPU Proof of Concept”
<code>--workdir PATH</code>	Output directory	<code>./amd-poc-YYYYMMDD-HHMM</code>
<code>--base-dir PATH</code>	Path to an existing <code>mariadb-tpcc-bench</code> checkout	auto-clone next to <code>workdir</code>
<code>--dry-run</code>	Show the plan, run nothing	off
<code>--yes / -y</code>	Non-interactive, accept defaults/flags	off
<code>-h / --help</code>	Show usage	—

Procedure — Running the Benchmark

Step 1: Copy the script to the benchmark host

```
scp amd-mariadb-poc-v1.2.sh user@benchmark-host:~/
ssh user@benchmark-host
chmod +x amd-mariadb-poc-v1.2.sh
```

Step 2: (Optional) Sanity-check the plan with a dry run

This walks through the wizard (or applies `--yes` defaults) and prints the run plan and time estimate without executing anything or touching host settings.

```
bash amd-mariadb-poc-v1.2.sh --dry-run --yes
```

Step 3: Run the interactive wizard (recommended for a customer PoC)

```
bash amd-mariadb-poc-v1.2.sh
```

The wizard will:

1. Print detected CPU model, socket/core/NUMA topology, RAM.
2. Ask for core counts to benchmark (e.g. `8 16 32 64`).
3. If the host has multiple NUMA nodes, ask whether to pin the run to one node (recommended when the largest core count fits in a single node).
4. Recommend and confirm a TPC-C warehouse count ($\geq 10\times$ the largest core count tested, to avoid hot-row contention masking true CPU scaling).
5. Ask for ramp-up/timed-run minutes, whether to also run TPC-H, InnoDB buffer pool size, and MariaDB version.
6. Ask whether to apply host tuning (governor, swappiness, hugepages) — needs `sudo`; all changes are reverted automatically when the script exits.
7. Ask for the customer name and report format, print the full run plan and a rough time estimate, then ask for final confirmation before starting.

Step 4: Or run fully non-interactively

Useful for scripted/repeatable PoCs, e.g. a 64-core EPYC PoC:

```
bash amd-mariadb-poc-v1.2.sh --yes \
  --cores "8 16 32 64" --warehouses 640 \
  --rampup 3 --duration 15 --buffer-pool 96G \
  --numa-node 0 --mariadb-ver 11.4 \
  --customer "Acme Corp" --report both
```

Step 5: Let it run

For each core count the script:

1. Starts background `mpstat` and RAPL power sampling.
2. Invokes the base runner (`mariadb-tpcc-bench.sh`) to build a fresh schema at that core budget and run the timed TPC-C (and optional TPC-H) test.
3. Stops sampling and computes average package power for that config.

Expect roughly (`schema-build-time` + `rampup` + `duration` + 5 min overhead) minutes **per core count** — the script prints its own estimate before starting. Schema build is ~15 seconds per warehouse and is repeated for every core count tested.

Step 6: Collect the report

On completion the script prints the output layout and consolidates everything into a report:

```
amd-poc-YYYYMMDD-HHMM/
amd-mariadb-poc-report.md   - consolidated PoC report (source of truth)
amd-mariadb-poc-report.pdf  - rendered via pandoc, if available
bench/                     - base runner workdir
  configs/                 *.cnf per core count
  results/                 raw HammerDB logs + per-run summaries
metrics/<N>core/           - power.txt (RAPL), mpstat.log
sysinfo/                   - lscpu, numactl, free, os-release, ...
```

Hand the customer `amd-mariadb-poc-report.pdf` (or `.md`); keep the rest for your own backup/troubleshooting.

PoC Tips

- **Warehouses $\geq 10\times$ the largest core count.** The wizard defaults to this. Too few warehouses causes hot-row contention that flattens the curve and undersells the CPU.
- **Pin to one NUMA node** with `--numa-node` when the largest core count fits in a single node — this removes cross-socket memory latency from the comparison. Check topology first with `numactl --hardware`.
- On EPYC, NUMA node cpulists enumerate physical cores before SMT siblings, so NUMA-derived cpusets naturally land on physical cores first.
- Use `--duration 15` or more for headline numbers; short (5-minute) runs are fine for dry-fitting the setup, but understate steady-state throughput.
- RAPL power needs readable `/sys/class/powercap/*rapl*` counters — run as root (or via `sudo`) if NOPM/Watt shows as - in the report.
- Durability is intentionally relaxed for peak-throughput measurement (`innodb_flush_log_at_trx_commit=0`, `innodb_doublewrite=0`); the report states this explicitly so the customer sees the methodology honestly.
- All host tuning is reverted automatically on exit, including on Ctrl-C.

Troubleshooting

Symptom	Likely cause / fix
Docker not found	Install Docker; the script exits immediately.
Docker daemon not reachable	<code>sudo systemctl start docker</code> , or add your user to the docker group.
libmariadb.so.3 missing	<code>sudo apt-get install -y libmariadb3</code> (Ubuntu/Debian) or <code>sudo yum install -y mariadb-connector-c</code> (RHEL/Rocky).

Symptom	Likely cause / fix
PDF not generated	<code>pandoc</code> or a LaTeX engine missing — install <code>pandoc</code> <code>texlive-latex-recommended</code> <code>texlive-fonts-recommended</code> , or re-run with <code>--report md</code> . Check <code>pandoc.err</code> in the workdir.
NOPM/Watt column shows -	RAPL energy counters not readable — re-run as root/sudo.
Container “did not start within 3 minutes”	Usually an invalid <code>--buffer-pool</code> value missing a unit suffix (e.g. <code>16</code> instead of <code>16G</code>) — MariaDB then requests far too little memory and fails to start.
Core count rejected	Requested core count exceeds host logical CPU count (<code>nproc</code>).

Reference

- Base benchmark runner: <https://github.com/skariapaul/mariadb-tpcc-bench>
- Workload: HammerDB TPC-C (OLTP) and optional TPC-H (analytics)
- Script version documented here: **v1.2**